

PROJECT PROPOSAL

Financial Statement Simulator

An auditable, standard-neutral simulator of corporate financial statements, with a validated reference implementation on real US GAAP and IFRS filings

Prepared by Jaebum (Albert) Chung | Prepared for Chak | Status: full proposal | July 2026

Internship window: 31 August 2026 to 26 February 2027

1. Executive summary

This proposal specifies an enterprise-grade financial statement simulator: a system that reads a company's annual report, re-expresses it in a standard-neutral economic state space, advances that state under a macro and industry scenario with an accounting engine that cannot break the books, and renders next-period statements in the company's own line items, under the company's own accounting standard. The design targets the uses a large bank has for such a machine: forward-looking credit assessment, scenario and stress analysis of counterparties and clients, portfolio-wide what-if screening, and a controlled analytical substrate that model risk management can actually audit.

Three design commitments distinguish the system from a spreadsheet model or a licensed data feed. First, *ingestion is held to a near-zero error bar* enforced by construction: several decorrelated readers extract every figure independently, a field is accepted only when independent paths agree exactly, and accounting identities arbitrate what survives; everything else is flagged for review, never guessed. Second, *the state-space encoding is lossless by theorem, not by hope*: encode-then-decode reproduces the source statement exactly, cell for cell, because the encoder is injective and the decoder recomputes exactly the values the accounting identities pin down. Third, *the simulation cannot silently break accounting*: every projected stock movement is carried by an explicit, balanced flow; cash is never a residual; the engine asserts the identities and rejects any path that violates them.

These are not aspirations. A validation build accompanying this proposal ran the full pipeline on four real annual reports, two US GAAP filers (Apple, Microsoft, Form 10-K) and two IFRS filers (SAP, Spotify, Form 20-F): **994 of 994** accepted PDF-extracted cells match the filers' own tagged values exactly (zero errors, zero silent misses, every discrepancy source explained); reconstruction through the state space is exact on **1,162 of 1,162** cells; and Monte Carlo simulation under six scenarios produces internally consistent statements on every path, moving in the economically required direction on every directional test. Section 10 reports the full results; the remaining sections specify the system, the mathematics it rests on, the testing regime, and the governance wrapper that makes it deployable inside a bank.

2. Context and alignment

The enterprise-grade bar for this project has been the intent from the outset. Prior intern contributions (Hong Xiao and Elton) established the core of the simulator: an internally consistent, no-circularity accounting engine and autoregressive driver modeling validated on synthetic data. What the project did not yet have, and what this proposal scopes, are the pieces the system hinges on: an extraction layer that reads real filings (US GAAP and IFRS) into structured statements, per-

standard encode and decode adapters that map those statements to and from a common state space, the state space itself, and a credible relationship layer that maps scenarios to the flow variables.

The work divides into two parts. **Part I**, the MVP and the focus of this proposal, is a faithful simulator: it reads a real filing into a structured statement and produces next-period statements that carry the same line items, comply with the source standard, and move in the right direction under scenarios. Its driver layer is a simple state-space model, stochastic by construction, so Part I already runs stochastic simulations: for a given scenario it samples driver noise and propagates it through the engine to a distribution of internally consistent statements. **Part II**, a later and conditional extension, keeps the same machinery but replaces the simple noise with a learned, conditional joint distribution of the drivers, the main modelling IP. In the phased plan of Section 13, Part I is Phases 0 through 3 and Part II is Phase 4.

Since the draft of this proposal, the riskiest mechanical claims have been retired by working code. A one-day knowledge-graph spike proved the taxonomy-graph encoding on one balance sheet; the validation build described in Section 10 extended that proof to all three core statements, four firms, two accounting standards, a PDF-only extraction mode with a measured error rate, exact reconstruction, and closed-book simulation under scenarios. The proposal can therefore commit to acceptance gates with observed evidence that they are reachable, rather than engineering judgment alone.

3. Problem statement and intended use

A bank consumes corporate financial statements at scale: in credit underwriting and annual reviews, counterparty monitoring, leveraged-finance covenant design, equity and debt research, and client advisory. Nearly all of that consumption is retrospective. The forward-looking questions that matter, what do this client's statements plausibly look like next year if rates stay high, if their industry's pricing tightens, if a recession arrives, are answered today by hand-built spreadsheet models: slow to build, inconsistent across analysts, unauditable in aggregate, and quietly dependent on plugs that make balance sheets balance by fiat.

The simulator addresses that gap with a system that is (i) *faithful*: it starts from what the company actually filed, to the digit; (ii) *consistent*: every simulated statement satisfies the accounting identities exactly, with no plugged balancing items; (iii) *comparable*: one common state space spans firms and standards, so the same scenario can be pushed through every name in a portfolio; (iv) *presentable*: outputs render in each firm's own statement lines, so an analyst or an accountant reviews familiar statements, not a model's internal schema; and (v) *auditable*: every number in every output traces to inputs, rules, and seeds that reproduce it bit for bit.

Intended users and uses. Credit and research analysts (single-name scenario statements), portfolio and risk teams (batch what-if screening under common scenarios), and stress-testing functions (a consistent counterparty-level complement to top-down models). The MVP is an analytical tool, not a regulatory model; the governance posture (Section 12) is nevertheless built so that graduation into a governed model inventory is a process step, not a rewrite.

Non-goals. The system does not forecast point estimates competitively with equity research, does not price securities, and does not attempt cross-standard conversion (a statement read under IFRS is simulated and rendered under IFRS). Part I's stochastic output is a scenario-conditioned distribution with reasoned dynamics, not a calibrated probabilistic forecast; calibration is exactly the Part II extension.

4. Prior work

Internal. Three internal lines of work feed this design directly, and the reference implementation adopts concrete machinery from each. The prior interns' simulator frameworks contribute the *symbolic-first* discipline: financial models declared as SymPy equation systems, validated before any numerics (DAG acyclicity, completeness, and the accounting identity proven by symbolic cancellation), then compiled into batched TensorFlow pipelines; this proposal's engine adds exactly that verification layer (Section 7) and runs its Monte Carlo in TensorFlow the same way. The author's prior LLM-extraction pipeline contributes the LLM calling logic (a gateway client with patient retry and tolerant JSON unwrapping), the page-identification and schema-constrained extraction prompt patterns, and the repeated-run median-vote hallucination control, all reused verbatim in the gated LLM stages of Section 9; its forecasting layer is an independent TensorFlow implementation of the Vélez-Pareja cash-budget ordering whose no-circularity treatment this engine matches. The director's extraction tooling established the redundant-reader pattern that remains the robustness backbone. A one-day spike (June 2026) validated the knowledge-graph encoding mechanics on a real balance sheet, and the validation build accompanying this proposal extended it end to end.

Statement modeling without plugs. The requirement that projected statements balance by construction rather than by a plug has a literature. Vélez-Pareja and Tham develop forecasting schemes with no plugs and no circularity, showing that a balancing item computed as a residual conceals errors that double-entry construction exposes [1]. Stock-flow consistent macro modeling (Godley and Lavoie) applies the same discipline economy-wide: every flow comes from somewhere and goes somewhere, and accounting matrices must close [2]. Penman's financial statement analysis frames the articulation of income, balance sheet, and cash flow that the engine enforces mechanically [3]. The engine in this proposal is the firm-level, line-item-level application of those principles, with the closure asserted programmatically on every simulated path.

Structured financial data and taxonomies. XBRL 2.1 and its calculation linkbases define machine-readable statement structure [4], extended by the newer Calculations 1.1 specification [5]; the FASB US GAAP taxonomy [6] and the IFRS Accounting Taxonomy [7] supply the concept ontologies (roughly 17,000 concepts each) that this proposal uses as the state space's node set. The SEC has required interactive-data filings since 2009 [8] and inline XBRL for operating companies since 2019 per the EDGAR Filer Manual [9]; ESMA's ESEF imposes the same for EU issuers [10]. The open-source Arelle platform [11] is the reference processor for these artifacts and is used by regulators; it is the tag-path reader here.

Document extraction. Table structure recognition from rendered documents is an active field with dedicated datasets and models (for example PubTables-1M [12]); financial-domain question answering datasets such as FinQA [13] and TAT-QA [14] document how error-prone numerical extraction from filings is for learned systems, which motivates this proposal's decorrelated-reader gate and identity checks rather than trust in any single reader, learned or deterministic.

Vendor data. Standardized products (Compustat and peers) reclassify native line items into fixed templates: a non-injective encoding that discards the presentation this system must reproduce, so they cannot serve as the primary representation regardless of accuracy. As-reported vendor feeds preserve native lines but are themselves extraction pipelines with their own error rates, coverage, latency, and licensing constraints. Most decisively, the capability under examination is ingestion of an arbitrary annual report; outsourcing it fails the test by construction. A licensed feed remains useful as an external adjudication reader (Section 9).

Governance standards. The design aligns to SR 11-7's model-risk expectations (docu-

mented methodology, effective challenge, outcome analysis) [15] and to BCBS 239’s risk-data principles (accuracy, integrity, completeness, timeliness, adaptability) [16], which is what “enterprise-grade” concretely means in Section 12.

5. Definition of success (acceptance bar)

Given a prior-period statement and a scenario, the system produces simulated next-period statements that:

- an accountant cannot identify as obviously wrong;
- preserve the same field set as the input statement (exact line-item parity);
- conform to the applicable standard, US GAAP or IFRS, matching the source filing; and
- respond in the correct direction to scenario changes.

Quantified gates. “Done” is a battery, not a sentiment. The gates below bind the MVP, and every one of them has already been met by the validation build on its four-firm set (Section 10):

1. **Extraction.** On the gating set, the PDF-only mode (tag path withheld) must show *zero* errors among accepted cells against verified ground truth, with every non-accepted cell flagged rather than silently dropped, and every unmatched ground-truth row assigned to a documented benign category. The measured claim is reported with its rule-of-three bound: N accepted cells and zero errors bounds the per-field error rate above by $3/N$ at 95% confidence [17]. The architectural target for silent *concordant* error, all readers agreeing on the same wrong number and the identities failing to catch it, is of order 10^{-8} per field (Section 9); four filings cannot measure that directly, so it is defended by construction and by seeded-error testing, and reported honestly as such.
2. **Reconstruction.** $D(E(s)) = s$ exactly, on every cell of every statement of every firm in the set: values, labels, order, signs, units, and precision attributes.
3. **Arithmetic.** Every derived subtotal foots from its children within the decimals-based tolerance $0.5 \cdot 10^{-d} \cdot (n + 1)$ for a value stated with `decimals` = d and n addends; $A = L + E$ holds; the cash flow ties to the balance-sheet cash delta.
4. **Simulation integrity.** Across all Monte Carlo paths of all scenarios, zero identity violations: the engine may not introduce any imbalance beyond the filer’s own printed rounding residual, which it must preserve exactly.
5. **Directional battery.** Scenario means move with the required signs (Section 8); the battery must pass in full.
6. **Plausibility.** Sampled statements pass automated accountant-style bounds (growth, margin, non-negativity, continuity), and a qualified accountant review of representative statements finds nothing obviously wrong. Reviewer sign-off is the human gate on top of the automated battery.

Precision asymmetry. Extraction is held to the near-zero bar because a misread figure invalidates everything downstream. Encoding is exact by construction. Simulation output needs to be solid, robust, and directionally credible rather than precisely forecast; its correctness dimension is internal consistency, which is absolute.

6. System architecture

The simulation core is common to both standards; standard-specific work sits at the boundaries. Figure 1 shows the pipeline; Table 1 lists the components and their validation status.

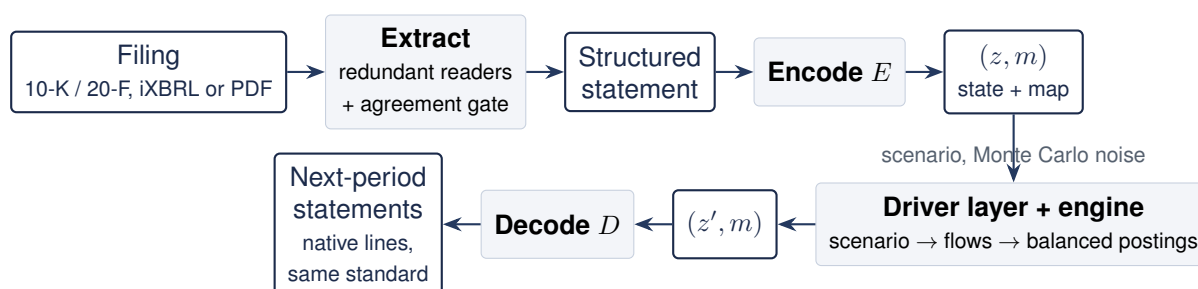


Figure 1. End-to-end pipeline. Extraction (reading the filing) and encoding (re-representing it) are distinct stages with different error models: extraction carries a measured error rate; E and D are deterministic and lossless. The presentation map m retained at encode time is what makes the output print in the firm’s own line items.

Component	Status	Role
Statement extractor	Validated	Reads a filing (tagged data or PDF) into a structured statement using decorrelated readers and an agreement gate; carries the near-zero read-error target.
Encode/decode adapters	Validated	Losslessly map structured statements to and from the common state space; perfect reconstruction proven per filing.
Common state space	Validated	Standard-neutral state over the US GAAP and IFRS taxonomies, specified as a knowledge graph (Section 6.2); retains the per-firm presentation map.
Accounting engine	Rebuilt + validated	Evolves stocks from balanced flows; identities preserved by construction, asserted on every path (Section 7).
Driver-to-flow layer	MVP validated	Maps scenarios to flows with reasoned nonlinear responses and sampled noise (Section 8); the Part II learning slot.

Table 1. System components. “Validated” means exercised end to end by the four-firm validation build of Section 10.

6.1 Encoding, decoding, and perfect reconstruction

Why operate in a state space at all? First, line items are heterogeneous: firms present the same economics under different labels, orderings, and aggregations, so machinery written against native lines fragments into per-firm variants; the common state gives the simulation one stable schema. Second, standards become cheap to add: the driver layer and engine wire once to the standard-neutral state, so a new standard costs an adapter pair. Third, it is what makes Part II estimable: pooling firms requires a shared representation. The price of this indirection is that nothing may be lost on the way in or out.

Write s for a structured statement: an ordered list of rows, each carrying a taxonomy concept, an optional dimensional qualification (axis and member, for statements that disaggregate a

concept on their face), a native label, sign convention, unit and precision, and one value cell per reported column. The encoder outputs a pair

$$E(s) = (z, m), \quad (1)$$

where z carries the values of the information-bearing rows and m carries everything else: row order, labels, signs, units, precision, sections, column periods, and the derivation structure (calculation arcs and dimensional aggregation) discovered from the filing. The decoder renders $D(z, m) = \hat{s}$. Perfect reconstruction requires $D \circ E = \text{id}$ on the domain of valid statements, which holds if and only if E is injective.

Proposition 1 (Lossless reduction) *Let z store one value per (concept, dimensions, period-role) key for every row not derivable from the others, and let m store every remaining attribute of s together with the derivation rules*

$$v(r, p) = \sum_{c \in \text{children}(r)} w_{rc} v(c, p) \quad \text{or} \quad v(r, p) = \sum_{r' \in \text{members}(r)} v(r', p) \quad (2)$$

for calculation parents and dimensional aggregates respectively. Then E is injective and D as defined by re-rendering m and recomputing (2) is a left inverse of E .

The proof is by construction: two statements agreeing on (z, m) agree on every stored cell and every attribute, and their derived cells are equal because (2) is a function of stored cells. The only values dropped are exact functions of kept ones, so the reduction is lossless. Two lessons from the validation build sharpen the “key” clause, and both are now regression tests. *Dimensions*: Apple and Microsoft disaggregate revenue and cost of sales by product/service axis on the face, so the same concept legitimately appears in several rows; keys must carry the dimensional qualification. *Period roles*: a cash flow statement displays the same cash concept twice, as beginning and ending balances distinguished only by the preferred-label period role; collapsing them breaks injectivity, exactly the failure mode Proposition 1 forbids. Where a filer’s printed subtotal differs from (2) because the filer rounded each line independently (observed at SAP: reported subtotals off by one million at millions precision), the row is *demoted to stored* and the discrepancy disclosed: reconstruction stays exact and the filing’s arithmetic inconsistency is surfaced rather than hidden.

In a forecast the engine advances the state, $z \mapsto z'$, and the output is rendered through the retained map, $D(z', m)$: exact line-item parity in a forecast is the same mechanism as reconstruction with z' in place of z .

6.2 The state space as a knowledge graph

The common state space is specified as a typed graph over the standard setters’ own machine-readable ontologies: the US GAAP and IFRS XBRL taxonomies [6, 7], roughly 17,000 concepts each. Nodes carry the attributes the simulator relies on and the taxonomy already provides: period type (instant versus duration, precisely the stock/flow distinction), balance polarity (debit or credit, the sign conventions), monetary flags, and labels. Edges come in five types: *composition* edges imported from the calculation linkbases (subtotal arithmetic, including the newer Calculations 1.1 arcs [5], which one of the four validation filers already uses); *dimensional aggregation* edges for face-of-statement axes; *laws of motion*, authored, recording which flows update which stocks and the cross-statement articulation; *label and synonym* edges, imported, powering the untagged-mode semantic mapper; and *driver attachments* from Section 8. The authored layers cover the simulated core, on the order of one to three hundred concepts, not seventeen thousand;

in the validation build the authored role table plus a calc-descendant inheritance rule (an extension concept inherits the role of the subtotal it feeds) classified every face line of all four filers with no per-firm code.

Encoding a statement builds a *firm overlay* on this graph. Each native line resolves to a node: directly when tagged with a standard concept; as an anchored child of its calculation parent when tagged with a firm extension (SAP's cloud revenue lines are extensions anchored to IFRS revenue); and, in the untagged PDF-only mode, through label-index candidates accepted only if the induced composition subtree foots to the printed subtotals and polarity matches the printed signs, with unresolvable lines flagged under the abstain rule. The firm's disclosed leaves become the entries of z ; subtotals and aggregates are marked derived and recomputed on decode; m decorates the same overlay with the exact native presentation. One graph then serves five consumers: extraction disambiguation, the adapters, the engine's stock-flow wiring, driver attachment, and (later) cross-standard correspondence.

Three cautions are built in, all encountered in practice during validation: filed calculation linkbases are treated as the per-firm arithmetic backbone but verified, never trusted (SAP's own subtotals violate them at printed precision); dimensional breakdowns are restricted to face-of-statement axes for the MVP (note-level axes leak into face concepts otherwise); and taxonomy versions are pinned per fiscal year.

7. The accounting engine: no plugs, no circularity

The engine's contract is that it *cannot* produce an unbalanced statement, and that nothing is plugged to make that true. Let the balance-sheet leaves be accounts $a \in \mathcal{A}$ with balance polarity $\beta_a \in \{+1 \text{ (debit)}, -1 \text{ (credit)}\}$. A *flow* f posts amounts $\phi_f(a)$ to a set of accounts; a flow is *balanced* when its debit-signed legs cancel:

$$\sum_{a \in \mathcal{A}} \beta_a \phi_f(a) = 0. \quad (3)$$

Stocks evolve only through flows, $s_{t+1}(a) = s_t(a) + \sum_f \phi_f(a)$, so by linearity the accounting identity residual

$$R_t = \sum_a \beta_a s_t(a) = \text{Assets} - \text{Liabilities} - \text{Equity} \quad (4)$$

is invariant: $R_{t+1} = R_t$ for any set of balanced flows. The engine asserts this invariance exactly on every simulated path. The base residual R_0 is zero for three of the four validation filers; for the fourth (SAP) the *printed* leaves round to a two-million-euro imbalance at millions precision, the filer's own rounding, which the engine preserves to the cent and reports, rather than absorbing into any account. That distinction, preserve the filing's residual, add exactly zero of your own, is the operational meaning of "no plugs."

Period cycle. Each simulated period runs: (1) project income-statement leaves from the driver draw; (2) set operating-stock targets (receivables, inventory, payables, deferred revenue and their kin) through the firm's *own* working-capital cash flow lines, so a stock moves only if the firm's cash flow statement has a line that carries its delta; (3) hold the discretionary schedule at reasoned levels (capital expenditure scaling with revenue, dividends growing with capped income growth, buybacks policy-scaled, debt issuance and repayment held at the filed schedule, lease payments held with additions imputed); (4) close income to retained earnings, route share-based compensation to paid-in capital, buybacks to treasury where the firm carries one; (5) apply the liquidity policy: excess cash above the operating target sweeps into the firm's own investment-purchase lines, a behavioral treasury rule, not an accounting plug, the books balance with or

without it; (6) recompute every derived row through the firm's calculation arcs; (7) assert the invariants: residual invariance, and the cash tie, ending cash equals beginning cash plus the recomputed net-change row of the firm's own cash flow statement.

Articulation. Cross-statement identities hold by construction because both sides come from the same flow: the depreciation add-back equals the amount removed from the capital pool; working-capital rows equal the negatives of their stock deltas; IFRS-style operating add-backs (tax expense, net finance items) mirror the projected income lines with their cash counterparts (taxes paid, interest received and paid) articulated so the accrual-cash gap is zero and the related stocks stay put. The validation build's four filers exercise materially different articulation styles (US GAAP indirect with supplemental cash disclosures; IFRS starting from profit after tax with add-backs and cash items inside the operating total) and both close exactly.

Symbolic verification before numerics. Following the prior interns' frameworks, the engine's flow system is verified *symbolically* before any simulation runs: per firm, every flow's legs are assembled as SymPy expressions over the firm's own accounts, and $\sum_a \beta_a \Delta_a$ must simplify to zero identically, for all parameter values and noise draws, not merely on sampled paths. A nonzero residual names the unbalanced flow symbols, localizing the specification error; the engine's computation order is likewise validated as a DAG with an explicit topological order, making no-circularity a checked structural property rather than a code-reading exercise. This check earned its place immediately: it exposed a latent gap (income attributable to non-controlling interests posted to cash without an equity counterpart for firms lacking an NCI equity line) that four clean test firms could never trigger numerically. The runtime pipeline is then the interns' sequence exactly: symbolic checking, TensorFlow execution, numerical checking.

Auditability. Every flow is journaled with its magnitude and its double-entry effect description; every run writes the journal, the input hashes, versions, configuration, and seed. Reruns are bit-identical. An auditor can take any simulated figure and walk it back to filed values and named rules.

8. The driver-to-flow layer and scenarios

Scenarios are vectors $x = (\Delta g^{\text{GDP}}, \Delta\pi, \Delta r, z_c, z_d)$: GDP growth deviation from trend (percentage points), inflation deviation (pp), a parallel short-rate shift (basis points), an industry competition shock (z-score), and a firm demand shock (z-score). The MVP driver map is reasoned and nonlinear where the economics demands it, with parameters documented, not fitted:

$$g_{\text{rev}} = w_m \hat{g} + \frac{1}{100} (\beta_G \Delta g^{\text{GDP}} + \lambda_\pi \Delta\pi) + \beta_d z_d - \kappa_r z_c + \varepsilon_g, \quad \varepsilon_g \sim \mathcal{N}(0, \sigma_g^2), \quad (5)$$

$$\Delta\left(\frac{\text{COGS}}{\text{Rev}}\right) = \kappa_m z_c + \lambda_c \frac{\Delta\pi}{100} + \varepsilon_m, \quad g_{\text{opex}} = \alpha g_{\text{rev}} + \lambda_o \frac{\Delta\pi}{100} + \varepsilon_o, \quad (6)$$

$$y' = y + \rho_A \frac{\Delta r}{10^4} \text{ (asset yields)}, \quad c' = c + \rho_D \frac{\Delta r}{10^4} \text{ (debt rates)}, \quad \rho_A > \rho_D, \quad (7)$$

with $\hat{g}$ the firm's own trailing revenue growth, w_m a mean-reversion weight, and $\lambda_\pi < \lambda_c$ (inflation passes into costs faster than into prices, so a pure cost shock compresses margins); effective tax rate clipped to [5%, 45%]; dividend growth capped at $\min(g_{\text{rev}}, 10\%)$ and floored at zero; buybacks halved in deep downturns; revenue growth floored at -35% . Monte Carlo runs use common random numbers across scenarios, so scenario mean differences measure the scenario response, not sampling noise. Firms without separate interest lines receive the net rate effect on their combined other-income line, sized by their own cash, securities, and debt balances. Where a driver has no line to act on, it inherits down the concept hierarchy of the knowledge graph.

For a fixed scenario, Monte Carlo sampling of $(\varepsilon_g, \varepsilon_m, \varepsilon_o)$ propagates through the engine, so

the output is a distribution over complete, internally consistent statements; the identity assertions run on every path. The stochastic simulation itself runs in TensorFlow, following the batched-tensor pattern of the internal forecasting stack: all paths of a scenario execute as one vectorized float64 pass over $[n_{\text{paths}}, n_{\text{accounts}}]$ tensors with stateless seeded draws, per-path identity residuals checked to the currency unit; the median and noise-free paths are then replayed bit-exactly through the Decimal engine, fed the very shocks TensorFlow drew, to produce the audit artifacts. An agreement test holds the two implementations together at 10^{-10} relative on replayed paths. The **directional battery** requires, on scenario means versus baseline: expansion raises revenue and net income; recession lowers both; competition lowers net income and compresses gross margin; inflation without demand lowers net income; and a rate hike moves net income with the sign of the firm's own net cash position, computed from its balance sheet, not asserted. Part II replaces (5)–(7)'s noise with a learned conditional joint distribution, keeping every other component fixed.

9. Extraction robustness

Extraction is held to a far higher precision bar than the rest of the system because a misread figure invalidates everything downstream. The approach mirrors the engineering already used in the director's own tools: several readers cross-check one another, and a field is accepted only when they agree.

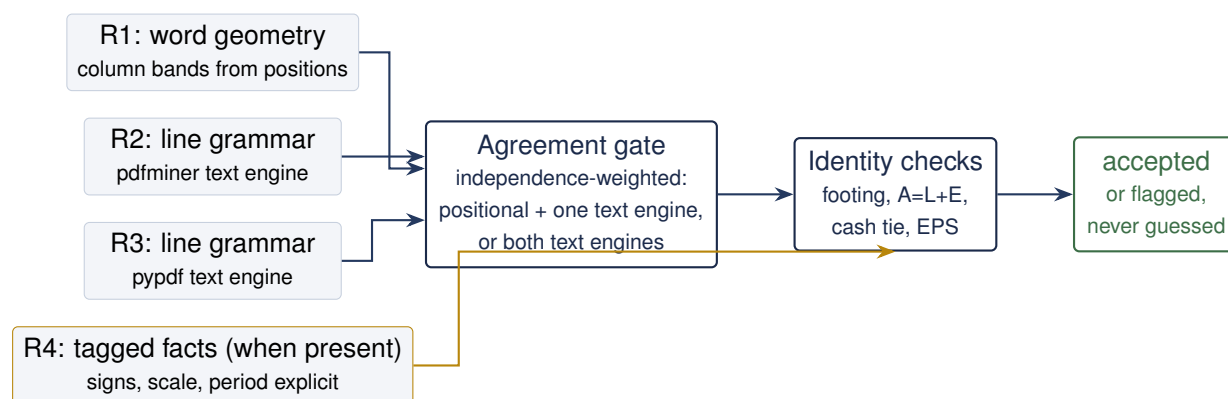


Figure 2. The reconciliation gate. Readers consume different representations of the same document; the gate weights agreement by path independence (the two line readers share a grammar, so their mutual agreement cannot outvote positional evidence); identity checks catch concordant errors that break arithmetic; tagged facts, when present, are the strongest reader and the ground truth for measuring the others.

Under this gating rule the two failure modes have very different costs: a disagreement costs review time; the expensive event is *silent concordance*, every reader returning the identical wrong value. With k genuinely independent readers of per-field error rates p_i and a collision kernel c (the probability that independent errors land on the same wrong value), the silent-concordance rate is of order $c \prod_i p_i$; with observed single-reader error rates at or below the percent level, decorrelated failure modes, and the identity filter removing arithmetic-breaking survivors, the design target is toward 10^{-8} per field, *measured* where measurement is possible and defended by construction where it is not. Three filters implement this:

1. **Cross-reader agreement** catches discordant errors; every disagreement is surfaced, never resolved silently. The acceptance rule is independence-weighted: the geometry reader plus either text-engine reader accepts; the two text-engine readers agreeing with each other, without positional support, is accepted only where geometry has no reading at all, and recorded

as the weaker rule.

2. **Decorrelated input paths:** word positions; two different text extraction engines under one line grammar; the filer's tagged facts when present (explicit sign, scale, period, no dependence on visual layout); optionally a vision reader and the prior year's comparatives. Layout pathologies (wrapped labels, note-reference columns, scale headers, glued currency amounts, unlabeled section totals) hit the layout readers and not the tag path; tag-path pathologies (extension tags, footnote-only values) do not afflict the layout readers.
3. **Accounting-identity checks** catch concordant errors that break arithmetic: footing along the calculation arcs, $A = L + E$, the cash tie, and per-share consistency, all within the decimals tolerance. They also catch genuine typos in the source filing, which are surfaced, since the extractor's ground truth is what the filing says.

Tags are the best case, not a prerequisite. Where a document carries no tags, the remaining readers still gate one another and the identities still bind; reduced redundancy surfaces as a higher flag rate, not silent error. Because the untagged mode is the one that generalizes beyond public filings, it is measured separately: the *PDF-only ablation* withholds the tag path from extraction entirely and scores the result against it. Section 10 reports that ablation at zero errors on 994 accepted cells across the four filers, and an untagged sweep over fourteen arbitrary investor-relations annual reports (US GAAP and IFRS, five jurisdictions, banks and industrials, five distressed filers).

The LLM's leash. LLM stages reuse the author's prior extraction pipeline wholesale: the calling logic, the batched page-identification prompts, the schema-constrained JSON extraction rules, and the repeated-run median-vote hallucination control. Their authority is deliberately narrow. Page identification is a fallback when the deterministic locator abstains. Cell reading operates only on cells the reader gate flagged, and a flagged cell is accepted only when the LLM's median-voted value exactly matches one of the deterministic readers, so the LLM can break ties but can never introduce a number nobody read. Concept mapping chooses from a lexically retrieved shortlist and is vetoed by balance-polarity checks. When no endpoint is configured every stage degrades to its deterministic behavior and the corresponding cells simply remain flagged; a firm whose balance sheet carries flagged cells is refused simulation outright (a structural quality gate), which is precisely the review loop the abstain rule intends.

LLMs at build time, determinism at run time. The leash above binds *what* the model may say; a second, stricter rule binds *when* it may be consulted at all. Production ingestion must replay exactly to pass model approval, and a hosted model in the inference path cannot meet that bar: inference kernels are load-dependent in their low-order bits, greedy decoding amplifies a last-bit difference into a different answer, and hosted endpoints update silently, leaving no version to freeze. Accordingly the model is confined to change management. `fss onboard` runs ingestion with the LLM engaged and emits a *mapping artifact* per document: statement page locations, label-to-concept choices with balance polarity, and flagged-cell adjudications, each entry the survivor of the mechanical validators, with the source document's hash, the code version, and a sign-off field. A human reviews and signs the artifact; it is versioned alongside the code. `fss runtime` then never constructs a model client: it re-extracts deterministically, replays an adjudication only when a deterministic reader still reads the signed value today (the artifact can break ties but cannot inject numbers into a changed document), re-runs every check, logs the source, code, and artifact versions it ran under, and refuses a document whose hash has drifted with "re-onboarding required." The reference implementation demonstrates the property directly: two independent runtime executions over the same filings produce byte-identical outputs (twelve

artifacts, zero differing hashes), the fourteen-document sweep reproduces its build-time results with zero model calls, and when the original hosted endpoint began refusing requests between build and demonstration, the runtime was indifferent – the artifacts were reconstructed from the committed build products without a model, which is the failure mode this split exists to absorb.

Residual risk. What remains silent is the intersection: errors concordant across decorrelated paths, identity-preserving, and within rounding tolerance, for example a uniform mis-scaling that multiplies every value identically (identities are scale-invariant, so only the tag path or a share-consistency check catches it). The seeded-error battery deliberately injects the correlated pathologies (wrapped labels, scale exceptions, note-reference columns, digit transpositions, dropped signs) and requires the gate to flag every one; a re-prompt of the same model on the same input is never counted as an independent check; and adjudication weight follows path independence, a standard-tag value outweighing two layout readers that agree with each other.

10. Validation evidence: four filers, two standards

Everything in this section was produced by the reference implementation accompanying this proposal (`src/fss`, roughly 5,000 lines of typed Python with a pytest suite and full audit manifests), run end to end on the most recent annual reports of Apple and Microsoft (US GAAP, Form 10-K) and SAP and Spotify (IFRS, Form 20-F). PDFs were rendered from the official EDGAR primary documents; the tag path (Arelle over the filers’ inline XBRL) supplies ground truth. Balance sheet, income statement, and cash flow statement, all reported columns.

Extraction (PDF-only ablation). Zero errors on every accepted cell, all four filers, all three statements, all columns:

File (standard)	compared	match	mismatch	missing	flagged
Apple (US GAAP)	198	198	0	0	0
Microsoft (US GAAP)	227	227	0	0	0
SAP (IFRS)	319	319	0	0	0
Spotify (IFRS)	250	250	0	0	0
Total	994	994	0	0	0

Table 2. PDF-only extraction versus tag-path ground truth. “Compared” counts accepted numeric cells; the rule-of-three 95% upper bound on the per-field error rate at 994/994 is 0.30%. Every unmatched ground-truth row falls in a documented benign category (values printed inside a label rather than as cells; derived subtotals the filer does not print, verified through their children).

The pathologies survived on the way to zero are the point of the exercise: wrapped labels with values on either the first or the continuation line; label-embedded amounts (“allowance of \$944 and \$830”); share counts in thousands beside dollars in millions on one statement; IFRS note-reference columns, both alphanumeric and bare-numeric; unlabeled section-total rows; superscript footnote digits fused to labels and to column years; dashes as explicit zeros; and negated-label sign conventions throughout the cash flows. Each was caught by reader disagreement or ground-truth mismatch during development, fixed as a general rule (never a firm-specific patch), and several are now seeded-error regression tests.

Reconstruction. $D(E(s)) = s$ exactly on all 12 statements, 1,162 cells: values, labels, order, signs, units, precision. SAP’s sixteen filer-rounded subtotals were demoted to stored values with the discrepancies disclosed, exactly as Section 6.1 specifies; the other three filers required none.

Footing and identities. Every derived cell of every extracted statement foots within the decimals tolerance; $A = L + E$ and the cash ties hold on all filers (SAP’s printed one-million rounding

differences pass under the tolerance and are reported).

Simulation. Six scenarios (baseline, expansion, recession, competition, rate hike, inflation), 500 Monte Carlo paths each, per filer: zero identity violations on all 12,000 paths; the engine preserves each filer's base rounding residual exactly and adds none. The directional battery passes in full for all four filers, including the net-cash-position-signed rate test. Representative simulated statements (median-net-income path per scenario), the per-path metric distributions, and the flow journals are written under `out/acceptance/` for accountant review; automated plausibility bounds (growth within $[-40\%, +60\%]$, gross margin within 15pp of base, non-negative cash, positive assets, exact continuity) pass on all representative and deterministic paths.

Untagged sweep (arbitrary IR documents). Beyond the gating set, the untagged pipeline ran over fourteen investor-relations annual reports with no XBRL anywhere: nine current large-caps (including a bank, and IFRS filers from Germany, France, and Hong Kong) and five distressed filers (Lehman 2007, SVB 2022, Bed Bath & Beyond 2022, Wirecard 2018, Evergrande 2022). These documents exhibit pathologies the EDGAR set never shows, each now handled by a general rule with a regression test: DOCX-exported PDFs that split digits inside numbers ("245, 122") and even inside words; PDFs whose space glyphs vanish entirely ("CONSOLIDATEDBALANCESHEET"); pages carrying two statements; statements spanning pages with repeated titles; numeric note-reference columns; bank vocabulary ("statement of financial condition") and IFRS wording ("statement of profit or loss"). Where extraction succeeds, printed-subtotal footing (with signed cascade discovery for income-statement chains), $A = L + E$, and the cash tie verify without any tags, and the discovered structure is itself the calculation tree the simulator uses; a structural quality gate refuses simulation while any balance-sheet cell remains flagged, which is exactly where the LLM adjudicator (or a human reviewer) enters. One document's financial pages contain no extractable text at all (image-based), the designed boundary for the optional OCR/vision reader. Per-document reports, provenance, and the sweep summary land under `out/untagged/`.

What this de-risks. The three claims the full build hinges on are now demonstrated on real filings rather than argued: the taxonomy graph carries what the simulator needs and real statements land on it without manual mapping; values and presentation separate losslessly, so simulation operates on z alone and re-renders natively; and the no-plug engine discipline survives contact with materially different real-world articulation styles under two standards.

11. Testing and acceptance plan

Testing operates at four levels, all automated except the last:

1. **Unit and property tests:** the number grammar (parentheses, dashes, unicode minus, glued currency, scale clauses), the display grammar (wrapped labels, unlabeled totals, section tracking), the gate's seeded-error battery (a corrupted reader must be flagged or outvoted by independent evidence, never silently accepted; correlated line-reader errors must not outvote positional evidence), encoder injectivity cases (dimensioned rows, period roles, filer-rounded subtotals), and driver response signs.
2. **Golden statement tests:** extracted statements for the gating set are frozen; any change in extraction output diffs against the golden copies cell by cell.
3. **The acceptance battery** (`python -m fss accept`): re-runs extraction accuracy, reconstruction, footing, the full Monte Carlo, directional and plausibility batteries, and writes the acceptance report and manifest. This is the go/no-go gate for any change, and its verdict is binary.
4. **Accountant review:** representative simulated statements per firm per scenario, rendered na-

tively, reviewed under the “not obviously wrong” standard; findings feed the plausibility battery as new automated bounds.

The gating set for the internship phases extends the validation set to roughly three filers per standard with one deliberately awkward inclusion each (a filer with discontinued operations mid-history, and a loss-making filer), plus one truly untagged document (an older or private-company annual report) to exercise the PDF-only semantic mapper without any same-filing metadata. Ground truth for the untagged document is hand-verified double-entry.

Every accepted-cell error found at any point is a sev-1 against the extraction layer: the fix must be a general rule, demonstrated by a new seeded-error test, never a filer-specific patch.

12. Governance, controls, and operations

Model risk alignment (SR 11-7). The system is documented as: purpose and use (Section 3), methodology and theory (Sections 6.1–8), data lineage (filing hashes to outputs), developmental evidence (Section 10), ongoing monitoring (the acceptance battery re-run on every change and on every new filing ingested), and known limitations (Section 14). Effective challenge is enabled by determinism: any reviewer can reproduce any number from the manifest.

Data quality (BCBS 239 posture). Accuracy and integrity are the extraction gates themselves; completeness is measured (coverage and flag rates per statement); timeliness is bounded by EDGAR availability plus pipeline runtime (minutes per filer); adaptability is the role-table and rule architecture, changes to which are code-reviewed and battery-gated.

Audit trail. Per run: SHA-256 of every input document, tool and package versions, configuration, seeds, per-field provenance (each reader’s raw token, the rule that accepted or flagged it), the encode demotion list, the flow journal per simulated path, and the validation verdicts. Retention follows the bank’s records schedule; nothing in the pipeline mutates its inputs.

Security and licensing. All processing is local; the only egress is fetching public filings from EDGAR with the mandated identification headers and rate limits. Dependencies are permissively licensed (Apache-2.0, MIT, BSD); no copyleft components. Taxonomy files are cached under their publishers’ terms and not redistributed. LLM-based readers, when added, run as one more gated reader with schema-constrained output and no authority to accept a field alone; the architecture neither requires nor trusts them.

Operations. The pipeline is a CLI (`fetch`, `extract`, `measure`, `accept`) suitable for scheduled batch execution; per-filer runtime is minutes on commodity hardware, dominated by document parsing. Failure modes are loud: a filing that cannot be located, parsed, or reconciled produces a structured error and no partial output.

13. Phased build plan and timeline

The validation build compresses risk out of the original phasing: what was Phase 1’s core uncertainty is now reference code. The internship phases harden and extend it to the full bar.

- **Phase 0 (2 weeks): scope lock and harness industrialization.** Fix the gating set (three filers per standard, one untagged document); freeze acceptance criteria with the director and the accountant reviewer; port the validation build’s battery into the team’s CI; golden statements recorded.
- **Phase 1 (7 weeks): extraction and adapters at the full bar.** Extend the readers (vision reader; prior-year comparatives as an additional path; the LLM semantic mapper for the un-

tagged document under the constraint-checked hypothesis rule); adjudication UI for flagged fields; the seeded-error battery grown to the full pathology catalogue; IFRS coverage hardened beyond the two validated filers. Gate: the two extraction gates of Section 5 on the full gating set, including the untagged document at its own measured bar.

- **Phase 2 (2 weeks): engine integration and manual toggles.** Wire the encoded state into the engine lineage the team maintains (reconciling with the prior interns' engine where it adds capability); expose flow parameters as reviewed toggles; end-to-end demo on the gating set. This phase is integration, made fast by the validated reference.
- **Phase 3 (3 weeks): drivers, scenarios, and review.** Macro layer consumed from the existing external model (Jodie's) with the documented interface; the directional battery extended with the accountant's scenario expectations; stochastic runs reviewed; plausibility bounds tightened from findings.
- **Hardening (4 weeks):** accountant sign-off cycles, adversarial filings (restatements, 52/53-week years, currency redenominations), performance, documentation to the governance standard of Section 12.
- **Phase 4 (5 weeks, conditional): learned driver distributions.** The Part II estimator, trained across firms in the common state space, validated on calibration; started only on MVP sign-off.

Phase	Focus	Window	Weeks
Phase 0	Scope lock, harness industrialization	31 Aug to 11 Sep 2026	2
Phase 1	Extraction and adapters at the full bar (US GAAP + IFRS + untagged)	14 Sep to 30 Oct 2026	7
Phase 2	Engine integration and manual flow toggles	2 Nov to 13 Nov 2026	2
Phase 3	Driver layer, scenarios, stochastic validation	16 Nov to 4 Dec 2026	3
Hardening	Accountant review cycles, adversarial filings, docs	7 Dec 2026 to 15 Jan 2027	~4
Phase 4	Learned driver distributions (conditional)	18 Jan to 26 Feb 2027	~5

Table 3. Timeline across the internship window. The validation build pulls roughly two weeks of risk out of the draft plan's Phase 1, reallocated to the untagged-document bar and hardening.

Key milestones. MVP feature-complete by 4 December 2026; accountant sign-off by 15 January 2027; Phase 4 begins only on sign-off.

14. Risk register

Risk	Rating	Mitigation / honest statement
Uniform mis-scaling in untagged mode (identity-invariant)	Medium	Share/EPS cross-checks break scale invariance; prior-year comparatives as a reader; flagged, not silent, when detectors disagree. Residual risk stated in every output.
Filer layouts outside the grammar (new pathology)	Medium	The grammar failed loudly, never silently, on every pathology met so far; flag rate is the alarm; each fix lands as a general rule plus a seeded test.
Driver realism judged insufficient by reviewer	Medium	Part I bar is directional credibility, not point accuracy; parameters documented and adjustable; Part II is the calibrated upgrade path.
Filing anomalies (restatements, 53-week years, redenominations)	Medium	Hardening-phase battery; statements are period-keyed so anomalies surface as explicit mismatches.
Dependence on prior interns' engine internals	Low	The validated reference engine stands alone; integration reconciles capabilities rather than assuming them.
Accountant availability for the human gate	Low	Automated plausibility battery keeps the loop moving; review batches scheduled in the hardening window.

Table 4. Principal risks. Extraction correctness is deliberately absent as a line item: it is the system's central control, gated continuously, not a residual risk to accept.

15. Open questions for alignment

The standards scope (US GAAP and IFRS), the precision asymmetry, exact line-item parity, the state-space specification, and the feasibility of the acceptance gates are settled, the last of these by evidence. Three items remain open:

1. The external macro model's (Jodie's) output interface and granularity, to be confirmed before Phase 3 wiring.
2. The reviewer and counts for the accountant gate: a provisional definition is in place (each gating-set firm, each scenario, representative paths); the named reviewer and batch sizes need confirmation.
3. Which mode gates Part I acceptance: tagged public filings or the untagged document at the same bar. The pipeline measures both; the recommendation is to gate on tagged filings and report the untagged bar alongside, tightening it to parity during hardening.

References

- [1] I. Vélez-Pareja and J. Tham, "Forecasting Financial Statements with No Plugs and No Circularity," *The IUP Journal of Accounting Research and Audit Practices*, 2010; and I. Vélez-Pareja, "To Plug or Not to Plug: A Note on Financial Statement Forecasting," SSRN Working Paper.
- [2] W. Godley and M. Lavoie, *Monetary Economics: An Integrated Approach to Credit, Money, Income, Production and Wealth*, Palgrave Macmillan, 2007.
- [3] S. Penman, *Financial Statement Analysis and Security Valuation*, 5th ed., McGraw-Hill, 2013.

- [4] XBRL International, *Extensible Business Reporting Language (XBRL) 2.1*, Recommendation, 2003 (with errata).
- [5] XBRL International, *Calculations 1.1*, Recommendation, 2023.
- [6] Financial Accounting Standards Board, *US GAAP Financial Reporting Taxonomy*, annual releases.
- [7] IFRS Foundation, *IFRS Accounting Taxonomy*, annual releases.
- [8] U.S. Securities and Exchange Commission, *Interactive Data to Improve Financial Reporting*, Release No. 33-9002, 2009.
- [9] U.S. Securities and Exchange Commission, *EDGAR Filer Manual, Volume II*, current version (inline XBRL requirements).
- [10] European Securities and Markets Authority, *European Single Electronic Format (ESEF) Regulatory Technical Standard*, in force 2020.
- [11] Arelle project, *Arelle: An Open Source XBRL Platform*, arelle.org; version 2.41 used in the validation build.
- [12] B. Smock, R. Pesala, and R. Abraham, "PubTables-1M: Towards Comprehensive Table Extraction from Unstructured Documents," *CVPR*, 2022.
- [13] Z. Chen et al., "FinQA: A Dataset of Numerical Reasoning over Financial Data," *EMNLP*, 2021.
- [14] F. Zhu et al., "TAT-QA: A Question Answering Benchmark on a Hybrid of Tabular and Textual Content in Finance," *ACL*, 2021.
- [15] Board of Governors of the Federal Reserve System and OCC, *Supervisory Guidance on Model Risk Management*, SR Letter 11-7, 2011.
- [16] Basel Committee on Banking Supervision, *Principles for Effective Risk Data Aggregation and Risk Reporting* (BCBS 239), 2013.
- [17] B. D. Jovanovic and P. S. Levy, "A Look at the Rule of Three," *The American Statistician*, 51(2), 1997.
- [18] J. Singer-Vine and contributors, *pdfplumber* (MIT license); and pypdf contributors, *pypdf* (BSD license): the two independent PDF engines of the validation build.
- [19] KG Encoding Spike report, this repository, June 2026: taxonomy-graph loading, balance-sheet overlay, footing, and round-trip on one filing.